

MSSQL FİNAL SINAVI ÇALIŞMA KONULARI (ARA SINAV KONULARINA EK OLARAK)

View Nedir? (DERSTEKİ SUNUMA EK OLARAK. AYRICA SUNUMDAN DA SORUMLUSUNUZ!!!)

View (Görünüm), bir veya birden fazla tablodan veri çekmek için oluşturulan sanal bir tablodur. Gerçek bir tablo değildir; içinde veri barındırmaz. Ancak bir tablo gibi sorgulanabilir.

Avantajları:

- Karmaşık sorguları basitleştirir.
- Güvenliği artırır (kullanıcının görmesini istemediğiniz kolonları gizleyebilirsiniz).
- Kod tekrarını azaltır.
- Raporlama işlemlerinde kolaylık sağlar.

```
CREATE VIEW view_adı AS
SELECT kolon1, kolon2, ...
FROM tablo
WHERE koşul;
```

Basit Örnek

Tablo: Öğrenciler

OğrenciID	Ad	Soyad	Bolum	Ortalama
1	Ahmet	Yılmaz	Bilgisayar	85
2	Ayşe	Demir	Elektrik	72
3	Mehmet	Kaya	Bilgisayar	90

Sadece Bilgisayar Bölümündeki Öğrencileri Gösteren View:

sql

```
CREATE VIEW BilgisayarOgrencileri AS
SELECT Ad, Soyad, Ortalama
FROM Ogrenciler
WHERE Bolum = 'Bilgisayar';
```

Bu view üzerinden artık şöyle sorgu yazabilirsiniz:

sql

```
SELECT * FROM BilgisayarOgrencileri;
```

View Güncelleme

Bir view güncellenmek istendiğinde:

sql

```
ALTER VIEW BilgisayarOgrencileri AS
SELECT Ad, Soyad, Ortalama, Bolum
FROM Ogrenciler
WHERE Bolum = 'Bilgisayar';
```

View Silme

sql

```
DROP VIEW BilgisayarOgrencileri;
```

MSSQL DE TRANSACTION

MSSQL'de TRANSACTION (işlem) konusu, bir grup SQL sorgusunun bir bütün olarak çalışmasını sağlamak için kullanılır. Yani bu sorgular ya tamamı başarılı olur ve veritabanına kalıcı olarak uygulanır, ya da herhangi biri başarısız olursa tümü geri alınır (rollback). Bu, özellikle para transferleri, sipariş işlemleri gibi kritik durumlarda veri tutarlılığını korumak için önemlidir.

Temel Komutlar

- **BEGIN TRANSACTION:** İşlemi başlatır.
- **COMMIT:** Tüm işlemleri kalıcı hale getirir.
- **ROLLBACK:** İşlem başarısızsa yapılan her şeyi geri alır.



Örnek 1: Basit Transaction (Başarılı senaryo)

sql

```
BEGIN TRANSACTION;
```

```
UPDATE Hesaplar
```

```
SET Bakiye = Bakiye - 500
```

```
WHERE HesapNo = 1001; -- A kişinin hesabından 500 TL çek
```

```
UPDATE Hesaplar
```

```
SET Bakiye = Bakiye + 500
```

```
WHERE HesapNo = 1002; -- B kişinin hesabına 500 TL yatır
```

```
COMMIT;
```

Bu işlemde, A kişisinden para çekilip B kişisine aktarılıyor. Her iki işlem de başarılıysa COMMIT edilir ve veriler kalıcı olur.

❌ Örnek 2: Hata durumunda Rollback

```
sql

BEGIN TRANSACTION;

UPDATE Hesaplar
SET Bakiye = Bakiye - 500
WHERE HesapNo = 1001;

-- Hatalı hesap numarası, bu işlem başarısız olur
UPDATE Hesaplar
SET Bakiye = Bakiye + 500
WHERE HesapNo = 9999;

ROLLBACK;
```

💬 İkinci işlemde hesap bulunamadığı için işlem başarısız olur ve ROLLBACK devreye girer. A kişisinden çekilen 500 TL de geri alınır, sistem eski haline döner.

⚠ Neden Önemlidir?

- Veri tutarlılığını korur.
- Yarım kalan işlemleri engeller.
- Özellikle birden fazla tabloya yazma işlemi varsa vazgeçilmezdir.

✅ TRY...CATCH ile Transaction Kullanımı

```
sql

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Hesaplar
    SET Bakiye = Bakiye - 1000
    WHERE HesapNo = 123;

    -- Hatalı HesapNo, bu satır hata verecek
    UPDATE Hesaplar
    SET Bakiye = Bakiye + 1000
    WHERE HesapNo = 9999;

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;

    PRINT 'Hata oluştu: ' + ERROR_MESSAGE();
END CATCH
```

🔍 Açıklama:

- **BEGIN TRY:** İşlemleri denemeye başlar.
- **BEGIN TRANSACTION:** Transaction başlatılır.
- İkinci UPDATE hatalı olduğu için CATCH bloğuna geçilir.
- **ROLLBACK** işlemiyle önceki değişiklikler geri alınır.
- **ERROR_MESSAGE()** fonksiyonu ile hata mesajı yazdırılır.

🧠 Avantajları:

- Kod daha güvenli hale gelir.
- Olası hatalarda sistem eski haline döner.
- Hatalar daha okunabilir ve kontrol edilebilir olur.

MSSQL DE SUBQUERIES (ALTSORGULAR)

MSSQL'de alt sorgu (İngilizcesi: subquery) veritabanı sorgularında çok sık kullanılan ve bir sorgunun içinde başka bir sorgu çalıştırmayı sağlayan güçlü bir yapıdır. Alt sorgular sayesinde, daha dinamik ve şartlı veri çekimleri yapılabilir.

Alt Sorgu Nedir?

Alt sorgu, başka bir sorgunun içinde yer alan **SELECT** ifadesidir. Genellikle şu amaçlarla kullanılır:

- Bir koşulu belirlemek
- Bir sütunun değerini karşılaştırmak
- Bir sonuç kümesi oluşturmak

Alt Sorgu Nerelerde Kullanılır?

Alt sorgular üç yerde kullanılabilir:

1. WHERE içinde
2. FROM içinde
3. SELECT içinde

1 WHERE İçinde Alt Sorgu (En yaygın)

```
sql

SELECT Ad, Soyad
FROM Ogrenciler
WHERE BolumID = (
    SELECT ID
    FROM Bolumler
    WHERE BolumAdi = 'Bilgisayar Programcılığı'
);
```

Açıklama:

"Bilgisayar Programcılığı" bölümünün ID'sini alt sorgu bulur, dış sorgu o bölüme kayıtlı öğrencileri getirir.

2 FROM İçinde Alt Sorgu

```
sql

SELECT BolumAdi, OgrenciSayisi
FROM (
    SELECT BolumID, COUNT(*) AS OgrenciSayisi
    FROM Ogrenciler
    GROUP BY BolumID
) AS AltSorgu
JOIN Bolumler ON AltSorgu.BolumID = Bolumler.ID;
```



Açıklama:

Alt sorgu her bölümdeki öğrenci sayısını bulur. Dış sorgu bu sayıları bölüm adlarıyla eşleştirir.

3 SELECT İçinde Alt Sorgu

```
sql

SELECT
    Ad,
    Soyad,
    (SELECT BolumAdi FROM Bolumler WHERE ID = Ogrenciler.BolumID) AS Bolum
FROM Ogrenciler;
```



Açıklama:

Her öğrencinin bölüm adını alt sorgu aracılığıyla getirir.

⚠ Dikkat Edilmesi Gerekenler

- Alt sorgular performansı etkileyebilir. Karmaşık sorgularda JOIN kullanımı daha verimli olabilir.
- = operatörü yalnızca tek değer dönen alt sorgularda kullanılmalıdır.
- Çoklu değer dönecekse IN, ANY, ALL gibi operatörler tercih edilmelidir.

MSSQL DECLARE

MSSQL'de DECLARE ifadesi, değişken tanımlamak için kullanılır. Programlama dillerinde bir değişken tanımlamak ne anlama geliyorsa, SQL Server'da da DECLARE ile aynı işi yapar: veri saklamak, üzerinde işlem yapmak ve sorgularda kullanmak.

DECLARE Nedir?

DECLARE, SQL Server'da bir değişken oluşturmak için kullanılır. Tanımlanan değişkene sonradan SET ya da SELECT ile değer atanabilir.

Temel Sözdizimi:

```
sql  
  
DECLARE @DegiskenAdi VeriTipi;
```

Temel Sözdizimi:

```
sql  
  
DECLARE @DegiskenAdi VeriTipi;
```

Değişkene Değer Atama

1. SET ile:

```
sql  
  
DECLARE @Sayi INT;  
SET @Sayi = 10;
```

2. SELECT ile:

```
sql  
  
DECLARE @Ortalama FLOAT;  
SELECT @Ortalama = AVG(NotOrtalamasi) FROM Ogrenciler;
```

KULLANIM ÖRNEKLERİ

Kullanım Örnekleri

Örnek 1: Sabit bir değeri yazdırma

```
sql

DECLARE @Mesaj NVARCHAR(100);
SET @Mesaj = 'Merhaba SQL!';
PRINT @Mesaj;
```

Örnek 2: Değişkenle koşullu sorgu

```
sql

DECLARE @BolumID INT;
SET @BolumID = 2;

SELECT * FROM Ogrenciler
WHERE BolumID = @BolumID;
```

Örnek 3: Matematiksel işlem

```
sql

DECLARE @Sayi1 INT, @Sayi2 INT, @Toplam INT;
SET @Sayi1 = 15;
SET @Sayi2 = 25;
SET @Toplam = @Sayi1 + @Sayi2;

PRINT 'Toplam: ' + CAST(@Toplam AS NVARCHAR);
```

Nerelerde Kullanılır?

- Stored procedure içinde
- Fonksiyonlarda
- Karmaşık sorgularda geçici değer tutmak için
- Döngülerde (WHILE) ve koşullarda (IF, CASE) kullanılır

MSSQL DE STORED PROCEDURE

MSSQL'de Stored Procedure (Saklı Yordam), önceden yazılmış ve veritabanında saklanan SQL komutları bloğudur. Bu yordamlar çağrıldıklarında bir veya daha fazla SQL işlemini çalıştırır.

Stored Procedure Nedir?

Bir Stored Procedure, sık kullanılan SQL işlemlerini tekrar tekrar yazmak yerine, bir kere tanımlanıp gerektiğinde çağrılmasını sağlar.

Tıpkı bir programlama dilindeki "fonksiyon" gibidir ama genellikle veritabanı işlemleri (INSERT, UPDATE, DELETE, SELECT vb.) için kullanılır.

Avantajları

- Kod tekrarını azaltır
- Bakımı kolaylaştırır
- Güvenliği artırır (SQL injection'a karşı koruma)
- Performansı artırır (derlenmiş halde saklanır)

Temel Yapı:

```
sql

CREATE PROCEDURE ProsedurAdi
AS
BEGIN
    -- SQL komutları buraya
END;
```

Örnek 1: Parametresiz Stored Procedure

Amaç: Tüm öğrencileri listele

```
sql

CREATE PROCEDURE sp_OgrenciListele
AS
BEGIN
    SELECT * FROM Ogrenciler;
END;
```

Kullanımı:

```
sql

EXEC sp_OgrenciListele;
```

Örnek 2: Parametrelili Stored Procedure

Amaç: Belirli bir bölüme ait öğrencileri getir

```
sql

CREATE PROCEDURE sp_BolumeGoreOgrenciListele
    @BolumID INT
AS
BEGIN
    SELECT * FROM Ogrenciler
    WHERE BolumID = @BolumID;
END;
```

Kullanımı:

sql

```
EXEC sp_BolumeGoreOgrenciListele @BolumID = 2;
```



Örnek 3: Kayıt Ekleme Stored Procedure

sql

```
CREATE PROCEDURE sp_OgrenciEkle
    @Ad NVARCHAR(50),
    @Soyad NVARCHAR(50),
    @BolumID INT
AS
BEGIN
    INSERT INTO Ogrenciler (Ad, Soyad, BolumID)
    VALUES (@Ad, @Soyad, @BolumID);
END;
```

Kullanımı:

sql

```
EXEC sp_OgrenciEkle @Ad = 'Ahmet', @Soyad = 'Demir', @BolumID = 1;
```

⚠ Dikkat Edilecek Noktalar

- Procedure adı sp_ ile başlamasa da olur, ama bu alışkanlıktır.
- Parametrelerin veri türleri ve isimleri dikkatle tanımlanmalıdır.
- İçinde BEGIN...END ile bloklar kullanılmalıdır.